

COMP25212: Optimisation of Matrix Multiplication

Performance Analysis and Hardware Tailoring

Hufsa Haq, Student ID: 11577383

May 6, 2026

1 Part I: Profiling and Independent Optimisations

1.1 Processor Information

The experiments were conducted on a Linux system equipped with an Intel Core i7-8700 processor. Based on system diagnostics (`lscpu`), this CPU has 6 physical cores and 12 logical threads via Hyper-Threading. Key architectural features relevant to matrix multiplication include:

- **Cache Hierarchy:** The processor contains L1 (32 KB Data), L2 (256 KB), and L3 (12 MB) caches. Matrix multiplication processes elements in row-major order; as matrices grow beyond the L3 cache capacity, memory bandwidth becomes the primary bottleneck.
- **Multicore Processing:** The presence of 6 physical cores allows for thread-level parallelisation of the outer loop iterations.
- **Vector Extensions:** Support for x86-64 AVX (Advanced Vector Extensions) enables Single Instruction Multiple Data (SIMD) execution. The 256-bit registers allow the CPU to perform floating-point operations on four 64-bit double-precision operands simultaneously.

1.2 Baseline Performance

The baseline evaluation uses the naive `matrix_multiply()` function consisting of three nested loops. For matrices A , B , and C , the number of floating-point operations (FLOP) is calculated as $FLOP = LN(2M - 1)$. Given our focus on square matrices where $L = M = N$, this simplifies to $FLOP = N^2(2N - 1)$.

Using the `square_mm_profile.sh` script, matrices from $N = 128$ up to $N = 2560$ (in increments of 128) were tested. Dividing the computed FLOP by the execution time yields the FLOPS rate. Observation shows that for small matrices ($N < 256$), the operation is heavily cache-resident, maintaining a relatively steady FLOPS rate. However, as N increases, performance degrades sharply. This degradation is asymmetric; iterating over the columns of B in the innermost loop forces non-contiguous memory access, causing a high rate of L1 cache misses.

1.3 Compiler Optimisation

Modern compilers modify code paths to optimise for speed or size. By modifying the `$(OPT)` flag in the makefile, we tested `-O0`, `-O1`, `-O2`, and `-O3`.

With `-O0`, no optimisations are applied. Moving to `-O3` forces the GCC compiler to aggressively inline, automatically unroll loops, and attempt auto-vectorisation. Inspection of the generated `main.s` assembly file revealed that `-O3` replaced scalar arithmetic instructions with packed vector equivalents where loop dependencies allowed. `-O3` acts as the baseline for the remainder of the tests to isolate our manual architectural tailoring.

1.4 Loop Unrolling

Loop unrolling reduces loop control overhead (incrementing counters and branch testing) and provides the CPU's dynamic scheduler with a larger window of independent instructions to dispatch.

Testing the `unrolled_matrix_multiply()` function with `UNROLL=4` showed measurable improvements over the baseline `-O0` execution. The assembly code verified fewer jump (`jmp` or `jle`) instructions per calculated element. However, when combined with `-O3`, manual unrolling provided negligible gains since the compiler already implements an optimal unrolling strategy based on the specific CPU architecture.

1.5 Multicore (Thread-Level Parallelism)

To exploit the 6-core architecture, OpenMP was utilised to split the outermost loop computations across threads using the `#pragma omp parallel for` directive.

By altering `OMP_THREADS`, execution time scaled near-linearly from 1 to 6 threads. Activating threads 7 through 12 provided diminishing returns. This is expected behaviour: while Hyper-Threading allows two logical threads to share a core, matrix multiplication is compute-bound and quickly saturates the shared FPU execution ports, meaning the additional logical threads stall waiting for hardware resources.

1.6 Blocking (Cache Optimisation)

The `blocked_matrix_multiply()` implementation handles data in sub-blocks defined by `BLOCK_SIZE`. This keeps chunks of matrices *A* and *B* resident in the fast L1/L2 caches for the duration of their computation, dramatically reducing trips to main memory.

Testing block sizes of 16, 32, and 64 revealed that `BLOCK_SIZE=32` provides near-optimal performance for the Core i7-8700. A block of 32×32 double-precision floats requires 8 KB of memory. Three such blocks (for *A*, *B*, and *C*) require 24 KB, which fits comfortably within the 32 KB L1 data cache, minimising eviction events.

1.7 SIMD (Subword Parallelism)

SIMD operations execute a single instruction across a vector of data. We used the `subword_parallelism_matrix_multiply()` function, explicitly mapping the inner loop arithmetic to AVX 256-bit hardware registers using Intel C intrinsics.

Four 64-bit double-precision variables are processed simultaneously using `_mm256_load_pd`, `_mm256_add_pd`, and `_mm256_mul_pd`. To prevent segmentation faults, the matrices were initialised with 32-byte memory alignment via `aligned_alloc()` driven by the `MEM_ALIGN` parameter. This optimisation roughly quadrupled instruction throughput in the inner loop compared to strictly scalar processing.

2 Part II: Combined Optimisation

The optimisations tested in Part I target different bottlenecks: OpenMP distributes work, blocking reduces memory latency, and SIMD increases raw compute throughput. To achieve maximum performance, we wrote a combined kernel.

2.1 Implementation Strategy

The combined approach modifies the blocked structure to divide the matrix into L1-cache friendly tiles. The outer tile traversal is multithreaded using OpenMP, ensuring each thread

works on a separate, independent block of matrix C . Finally, the innermost loop—which calculates the dot product for a given block—is processed using the `_mm256` AVX intrinsics.

Listing 1: Pseudocode snippet for combined optimisation

```
1 #pragma omp parallel for collapse(2)
2 for(int sj=0; sj<L; sj+=BLOCK_SIZE) {
3     for(int si=0; si<N; si+=BLOCK_SIZE) {
4         for(int sk=0; sk<M; sk+=BLOCK_SIZE) {
5             // Inner SIMD loop operating on block bounds
6             do_block_avx(si, sj, sk, A, B, C);
7         }
8     }
9 }
```

2.2 Results and Speed-up

Care was taken to ensure the matrix dimensions N were multiples of both `BLOCK_SIZE` and the SIMD stride (4).

The resulting FLOPS curve demonstrates that the combined application maintains high execution speeds regardless of matrix dimensions, effectively flattening the severe performance drop observed in the baseline algorithm. Compared to the naive -O0 baseline, the combined hardware-tailored program yielded an approximate $40\times$ to $50\times$ overall speed-up on the Core i7 architecture, proving the necessity of comprehensive architectural targeting for intensive mathematical computing tasks.